Threads

Lightweight units of execution within a process.

Disclaimer

This material has been developed exclusively for educational and academic purposes under the KPIT APEX Lab – Academic CSR Initiative. It is intended strictly for student learning, demonstrations, and classroom use.

Users are advised to verify and confirm the correctness of the contents before use.

It must not be copied, forwarded, shared, reproduced, or redistributed to any other individuals, groups, institutions, or platforms without prior written permission.

All examples, commands, diagrams, and demonstrations included here are meant solely to support conceptual understanding and learning.

Any use of these contents in commercial products, paid workshops, training programs, or proprietary documentation is strictly prohibited.

What is a Thread?

A thread is a lightweight execution unit inside a process

Multiple Threads share:

- Address space
- File descriptors
- Heap
- Global variables (.data/.bss).

Threads have their own:

- Stack
- Registers
- Thread ID
- signal mask
- scheduling attributes

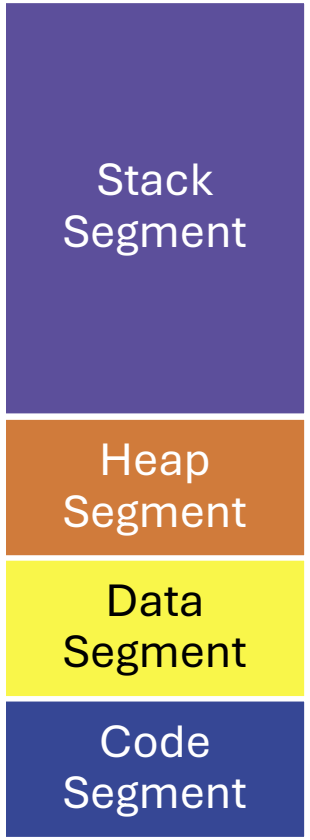
When to Use Threads

- Concurrent overlapped I/O (e.g., sensors, logging)
- Background tasks (telemetry, watchdog pings)
- Producer/consumer pipelines (ring buffers)
- Real-time task separation (with priorities & policies)

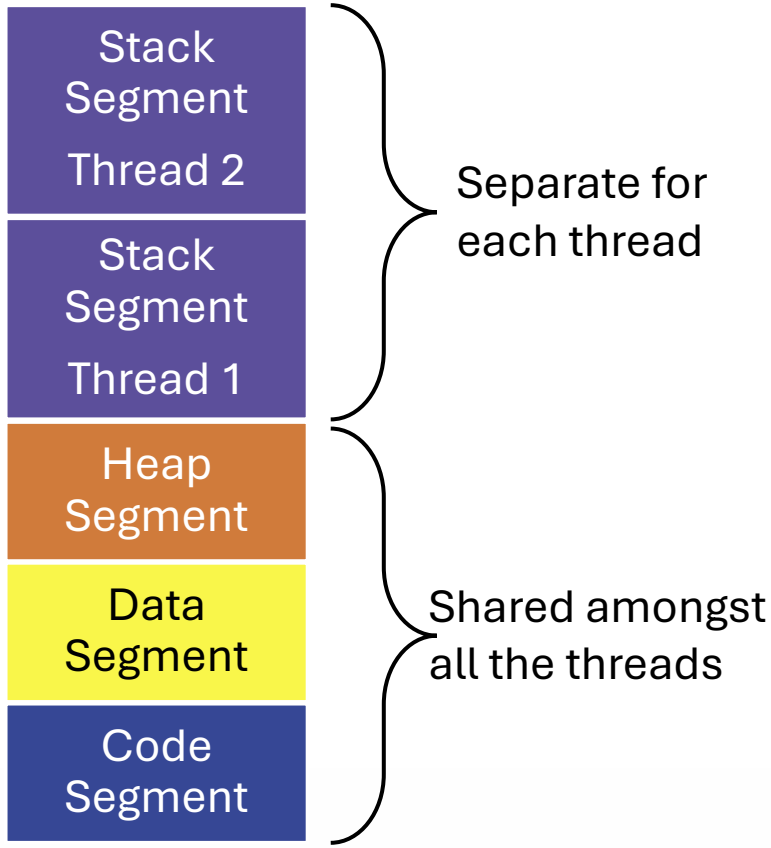
Use threads when sharing memory is necessary; otherwise consider processes for isolation.

Process – Thread Memory

Each process has a separate memory allocated.



A process can have multiple threads inside it.



Core pthread APIs

Syntax	Parameters	Return & Notes
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg)	thread: out param; attr: NULL for defaults; start_routine: function; arg: user data	0 success; non-zero error (e.g., EAGAIN). New thread starts at start_routine.
int pthread_join(pthread_t thread, void **retval)	thread: target; retval: receives return from pthread_exit() or function return	Blocks until thread terminates. ESRCH/EINVAL/EDEADLK possible.
void pthread_exit(void *retval)	retval: return pointer for joiner	Does not return. Cleans up thread resources properly.
int pthread_detach(pthread_t thread)	Make thread unjoinable; resources auto-released at exit	Use for fire-and-forget workers. Don’t join() after detach.

Thread ID

- Each thread within a process is uniquely identified by a thread ID
- This thread ID is returned by the caller of pthread_create()
- A thread can obtain its thread id by using pthread_self()

```
#include <pthread.h>
pthread_t pthread_self(void);
```

Creating simple Threads

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void* incrementFunction(){
for (int i = 0; i < 5; i++) {
        printf("Thread: i = %d\n", i); sleep(1);
}
return NULL; // Thread ends
}

void* decrementFunction(){
for (int j = 5; j > 0; j--) {
        printf("Thread: j = %d\n", j); sleep(1);
}
return NULL; // Thread ends
}
```

```c
int main(void){
        pthread_t thread_id1, thread_id2;

        pthread_create(&thread_id1,NULL,
        incrementFunction,NULL);
        pthread_create(&thread_id2,NULL,
        decrementFunction,NULL);

        pthread_join(thread_id1,NULL);
        pthread_join(thread_id2,NULL);

}
```

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void *thread_func_T2(void *arg) {
	printf("T2: Starting work...\n");
	sleep(2);
	printf("T2: Work completed.\n");
	return (void *)"T2 done";
}

void *thread_func_T1(void *arg) {
	pthread_t *t2 = (pthread_t *)arg;
	void *retval;
	printf("T1: Waiting for T2 to finish...\n");
	pthread_join(*t2, &retval);
	printf("T1: T2 finished with message: %s\n",(char *)retval);
	printf("T1: Doing its own work...\n");
	sleep(1);
	printf("T1: Work completed.\n");
	return (void *)"T1 done";
}

void *thread_func_generic(void *arg) {
	int id = *((int *)arg);
	printf("T%d: Doing work...\n", id);
	sleep(1);
	printf("T%d: Work completed.\n", id);
	return (void *)"Generic done";
}
```

```c
int main() {
	pthread_t t1, t2, t3, t4;
	int id3 = 3, id4 = 4;
	void *retval;

	// Create T2 first
	if (pthread_create(&t2, NULL, thread_func_T2, NULL) != 0)
		{perror("Failed to create T2"); exit(EXIT_FAILURE);}

	// Create T1 and pass T2's ID to it
	if (pthread_create(&t1, NULL, thread_func_T1, &t2) != 0)
		{perror("Failed to create T1"); exit(EXIT_FAILURE);}

	// Create T3 and T4
	pthread_create(&t3, NULL, thread_func_generic, &id3);
	pthread_create(&t4, NULL, thread_func_generic, &id4);

	// Main thread joins T1, T3, and T4
	pthread_join(t1, &retval);
	printf("Main: T1 finished with message: %s\n", (char *)retval);

	pthread_join(t3, &retval);
	pthread_join(t4, &retval);

	printf("Main: All threads completed. Exiting.\n");
	return 0;
}
```

Aspect	Process	Thread
Basic Definition	A process is an independent program in execution with its own memory space.	A thread is a lightweight unit of execution within a process.
Memory Space	Has its own separate virtual memory space.	Shares the process's memory space (text, data, heap).
Stack	Each process has its own independent stack.	Each thread has its own stack within the same process.
Creation Overhead	High – memory allocation, page tables, resources must be created.	Low – shares most resources, only stack + thread control block created.
Context Switch Cost	Higher – switching memory context and kernel structures.	Lower – same memory space, lighter task switch.
Communication	Requires IPC mechanisms (pipes, shared memory, message queues).	Very fast – communicates via shared memory within the process.
Isolation	Strong isolation – one process crash does not affect others.	Weak isolation – thread crash can terminate the entire process.
Resource Sharing	Not shared by default; must use explicit IPC.	Threads share code, data, heap, file descriptors.
Scheduling Unit in Linux	The process's main thread is scheduled as a task.	Each thread is scheduled as an individual task.
Concurrency	Limited by heavier switch + isolation boundaries.	High concurrency with lightweight switching.
Security	More secure due to isolation.	Less secure because shared address space.
Suitable For	Large independent applications, services, fault isolation.	Parallel tasks within the same app, real-time workflows, worker pools.
Linux Representation	Represented by a task_struct with unique PID.	Represented by task_struct with unique TID but same thread group ID.
Creation APIs	fork(), exec()	pthread_create() or clone() with flags.
Failure Impact	Process crash usually does not affect other processes.	Thread crash can bring down the entire process.
Typical Use in Embedded Linux	Running separate services, daemons, system processes.	Real-time tasks, communication handlers, worker threads, parallel computations.

Data Races & Synchronization

- **Data race**: two+ threads access same variable, at least one write, without synchronization → undefined behavior.
- Use **mutex** (pthread_mutex_t) to protect shared state.
- Use **condition variables** to wait/notify.
- Keep critical sections **small**; avoid deadlocks (consistent lock ordering).

pthread Attributes & Scheduling

- pthread_attr_t: set **stack size, detach state, inherit scheduler**.
- Real-time policies (if permitted): **SCHED_FIFO / SCHED_RR** via pthread_setschedparam.
- **Affinity** (pin threads to cores) with pthread_setaffinity_np (Linux-specific).
- **Guard size** to detect stack overflow (pthread_attr_setguardsize).

Safe Thread Exit & Cleanup

Prefer **cooperative shutdown**: set a flag; thread checks and exits.

pthread_cleanup_push/pop for resource cleanup on cancellation.

Detached threads: use for fire-and-forget jobs; **don't join** later.